



It's Your Life

with





기존 JS 의 이벤트 접근 방식



기존의 JS 가 HTML 요소를
컨트롤 하는 방법은

“이벤트 기반” 방식이었습니다!

즉, 사용자의 상호 작용(이벤트)가
발생하면 해당 이벤트를 기반으로
개발자가 HTML DOM 을
직접 선택하고 어떻게 변경할지
“명시적”으로 명령을 내려야 했습니다!



```
<!DOCTYPE html>  
<html lang="en">
```

```
<body>
```

```
<h1 class="header">Hello, Protocol</h1>
```

```
<button onclick="fetchData()">백엔드 통신 경험하기</button>
```

```
</body>
```

```
<script>
```

```
const headerEl = document.querySelector('.header');
```

```
async function fetchData() {
```

```
  try {
```

```
    const res = await fetch('http://localhost:4000/request');
```

```
    const data = await res.json();
```

```
    headerEl.innerHTML = data;
```

```
  } catch (err) {
```

```
    console.log(err);
```

```
  }
```

```
}
```

```
</script>
```

```
</html>
```

사용자에 의한
클릭 이벤트 발생

변경할 HTML 요소를
직접 선택

변경을 직접
"명시적"으로 지시



어떤 이벤트에 변경할지
안정함?

HTML 요소 선택 안함?

뭘 어떻게 변경할지
명령 안함?

개발자





개발자

스트레스

스트레스

잔소리 웅단 폭격





데이터 기반

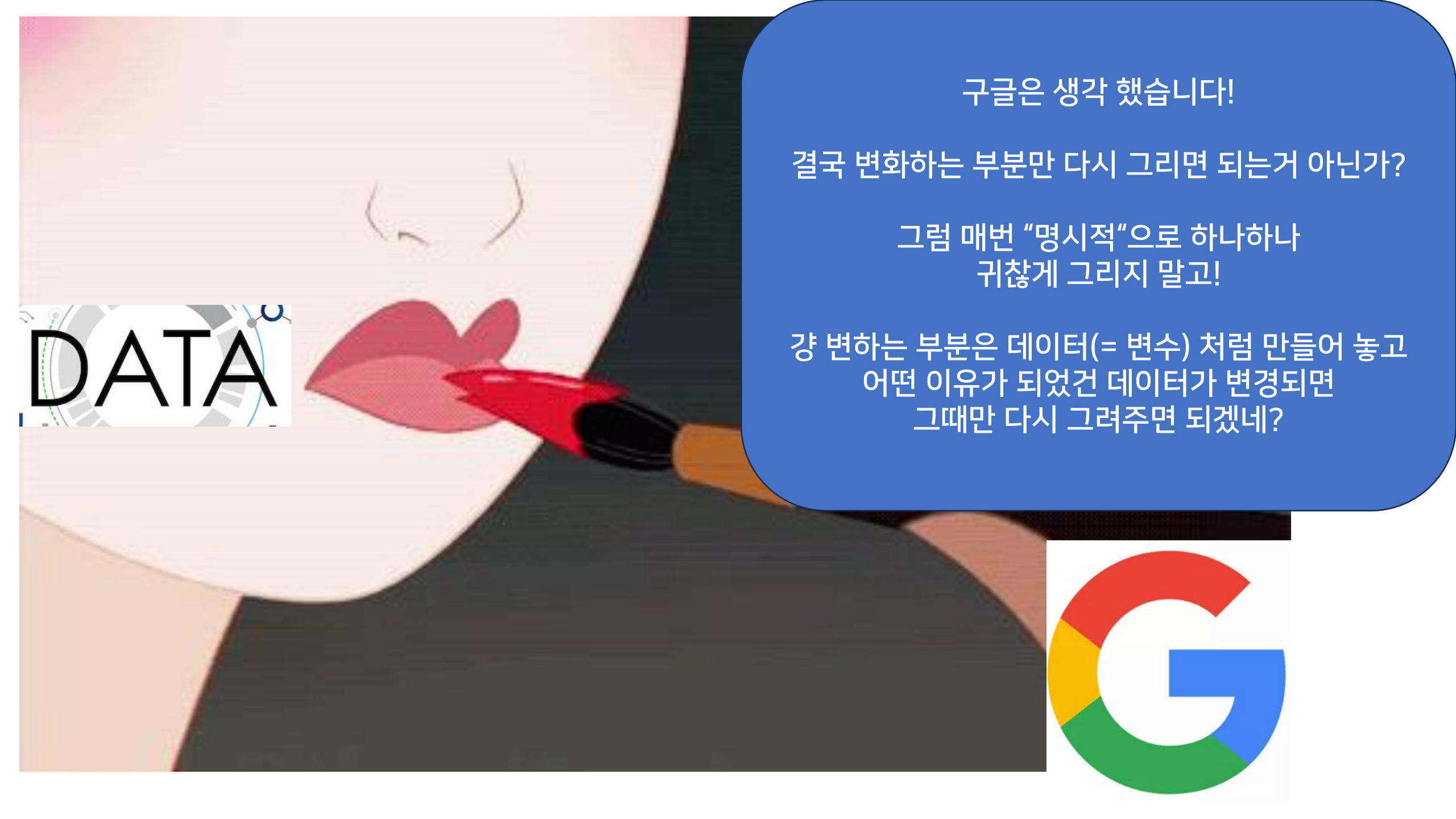
접근 방식의 이해



마루아 마루의 씨구

산은 산이요





DATA

구글은 생각 했습니다!

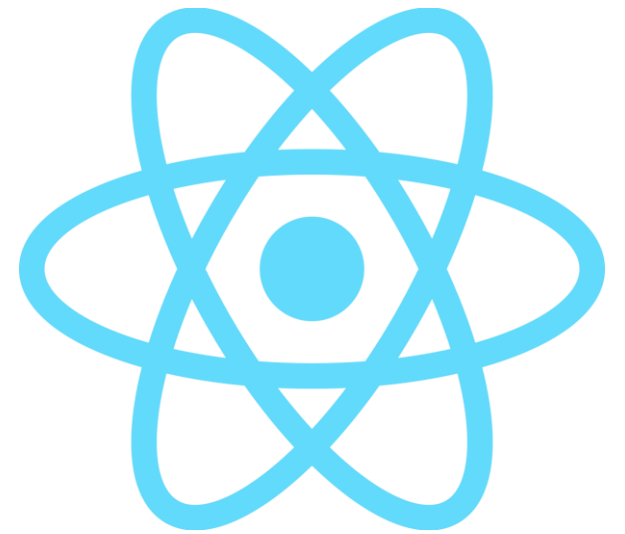
결국 변화하는 부분만 다시 그리면 되는거 아닌가?

그럼 매번 "명시적"으로 하나하나
귀찮게 그리지 말고!

강 변하는 부분은 데이터(= 변수) 처럼 만들어 놓고
어떤 이유가 되었건 데이터가 변경되면
그때만 다시 그려주면 되겠네?











코드로 확인!



```
<body>
  <div id="app">
    <h1>{{ count }}</h1>
    <button v-on:click="countUp">UP</button>
    <button v-on:click="countDown">DOWN</button>
  </div>
</body>
<script>
  const { createApp, ref } = Vue;
  createApp({
    name: 'App',
    data() {
      return {
        count: 0,
      };
    },
    methods: {
      countUp() {
        this.count++;
      },
      countDown() {
        this.count--;
      },
    },
  }).mount('#app'); </script>
```

data() 내부에 return 되는 변수들은
데이터 기반 접근을 위한 데이터로 사용!

→ 변화가 자동으로 추적되고, 변화가
발생하면 해당 부분이 다시 그려집니다(= 리렌더)!

```
<body>
  <div id="app">
    <h1>{{ count }}</h1>
    <button v-on:click="countUp">UP</button>
    <button v-on:click="countDown">DOWN</button>
  </div>
</body>
```

2개의 버튼을 이용하여
count 데이터를 각각
+1, -1 하는 메서드를
click 이벤트로 설정!

```
<script>
const { createApp, ref } = Vue;
createApp({
  name: 'App',
  data() {
    return {
      count: 0,
    };
  },
  methods: {
    countUp() {
      this.count++;
    },
    countDown() {
      this.count--;
    },
  },
}).mount('#app'); </script>
```

this 를 사용 data 에 접근하여
데이터의 변경을 발생



0

UP DOWN

6

UP DOWN

-4

UP DOWN

data 변경에 따라
화면이 바로 re-render 되어
다시 그려지는 것을 확인 가능!



EVENT

이벤트

명시적 명령

직접 DOM 선택

귀찮음

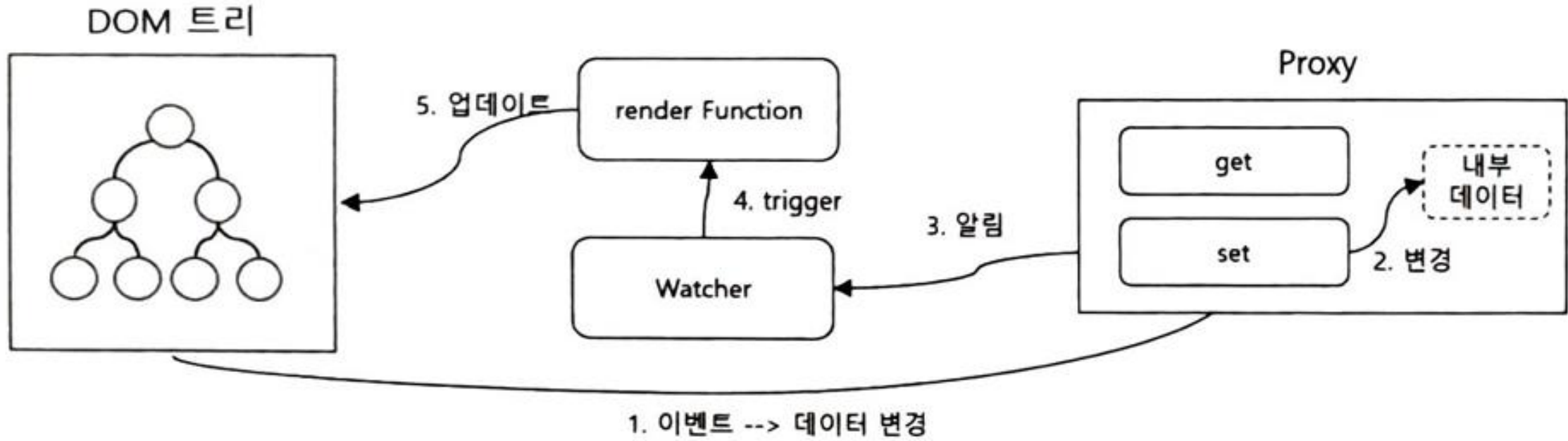
코드 길어짐

집에 가고 싶음

DATA



루트 Vue 인스턴스





DATA

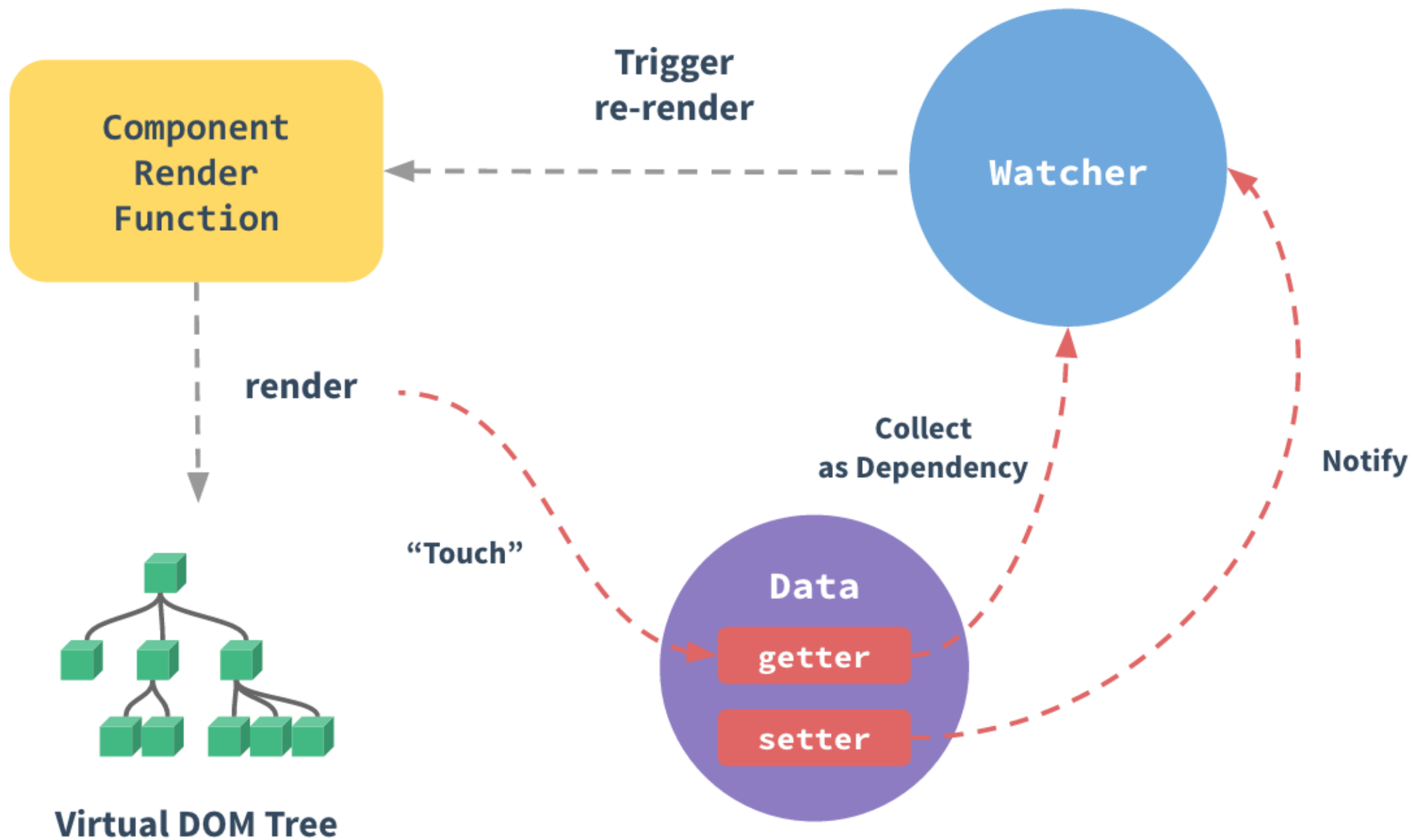




Proxy 는 Data 를 바라보고 있다가
Setter 가 호출이 되면

데이터 변경 이후에
해당 데이터 작업을 가로채서
화면을 리렌더링 시켜줍니다!

→ 데이터 반응형의 완성





어때요?
참 쉽죠?





Vue

이벤트



이벤트 핸들링

이벤트 리스닝하기

일반적으로 `v-on` 디렉티브는 단축 문법으로 `@` 기호를 사용하며, DOM 이벤트를 수신하고 트리거될 때, 사전 정의해둔 JavaScript 코드를 실행할 수 있습니다. `v-on:click="handler"` 또는 줄여서 `@click="handler"` 와 같이 사용합니다.

핸들러 값은 다음 중 하나일 수 있습니다:

1. **인라인 핸들러:** 이벤트가 트리거될 때 실행되는 인라인 JavaScript(네이티브 `onclick` 속성과 유사).
2. **메서드 핸들러:** 컴포넌트에 정의된 메서드 이름 또는 메서드를 가리키는 경로.



인라인

이벤트

인라인 이벤트 왜 쓰나요!?



- JS 에서 콜백 함수로 익명 함수를 바로 지정해서 보내주던 것 기억 하시나요?
- 비교적 간단한 작업을 위해 별도의 메서드를 선언하면 귀찮기 때문에 바로바로 지정해서 사용하기 위해 사용합니다!



귀찮





인라인 이벤트 사용하기!

```
<button v-on:click="count++">+</button>
```

v-on 디렉티브에
click 이벤트를 연결

인라인 스타일로
바로 JS 내용을 적용



```
<body>
  <div id="app">
    <h1>{{ count }}</h1>
    <button v-on:click="count++">+</button>
    <button v-on:click="count--">-</button>
  </div>
</body>
<script>
  const vueApp = Vue.createApp({
    name: 'App',
    data() {
      return { count: 0 };
    },
  }).mount('#app');
</script>
```

0



버튼을 클릭하면 v-on:click 에 의해
이벤트가 발생

지금은 인라인 이벤트로 지정이 되어
있으므로 count++ 또는 count-- 수행

count++ 또는 count-- 에 의해
count 데이터가 변화하고
렌더링이 발생!



이벤트 핸들러



이벤트 핸들러 왜 쓰나요!?

- 자 아래의 코드를 봅시다!!

```
<h1>{{ count }}</h1>  
<button  
  v-on:click="count = (parseInt(parseInt(Math.random() * 10) + 1) % (parseInt(Math.random() * 10) + 1))"  
>  
  count 를 랜덤한 숫자로 만든 다음 랜덤한 숫자로 나눈 나머지  
</button>
```




```
<h1>{{ count }}</h1>
<button
  v-on:click="count = (parseInt(parseInt(Math.random() * 10) + 1) % (parseInt(Math.random() * 10) + 1))"
>
  count 를 랜덤한 숫자로 만든 다음 랜덤한 숫자로 나눈 나머지
</button>
```

일단 코드가 잘 읽히나요!?

만약에 저런 코드를
다른 곳에서 1000000 번 불러야 한다면!?

그런데 갑자기 1000000 번 붙여 넣은
코드를 수정해야 한다면!??



```
<body>
```

```
  <div id="app">
```

```
    <h1>{{ count }}</h1>
```

```
    <button v-on:click="randRemain">
```

count 를 랜덤한 숫자로 만든 다음 랜덤한 숫자로 나눈 나머지

```
  </button>
```

```
</div>
```

```
</body>
```

```
<script>
```

```
  const vueApp = Vue.createApp({
```

```
    name: 'App',
```

```
    data() {
```

```
      return { count: 0 };
```

```
    },
```

```
    methods: {
```

```
      randRemain() {
```

```
        this.count =
```

```
          parseInt(parseInt(Math.random() * 10) + 1) %
```

```
            (parseInt(Math.random() * 10) + 1);
```

```
      },
```

```
    },
```

```
  }).mount('#app');
```

```
</script>
```

선언한 메서드를 이벤트에 연결

복잡한 부분은 따로 메서드로 선언!
JS 에서 마치 반복적인 작업을 함수로
선언해서 사용한 것과 정확히 동일 합니다!





실습, 이벤트 핸들러 활용하기

- 아래의 코드를 활용하여 페이지 생성 시, 생성되는 랜덤 한 숫자(0 ~ 20)를 맞추는 프로그램을 완성하세요!
- 확인 버튼을 누르면 text-input 입력 된 숫자 값과 랜덤 숫자를 비교해서 랜덤 숫자가 더 클 경우 Up 을, 작을 경우 Down 을 출력합니다
- 두 숫자가 같을 경우 '정답입니다' 를 출력하는 Vue App 을 완성하세요!
- v-if / v-show 디렉티브가 아닌 이벤트 핸들러를 통해 구현해 주세요



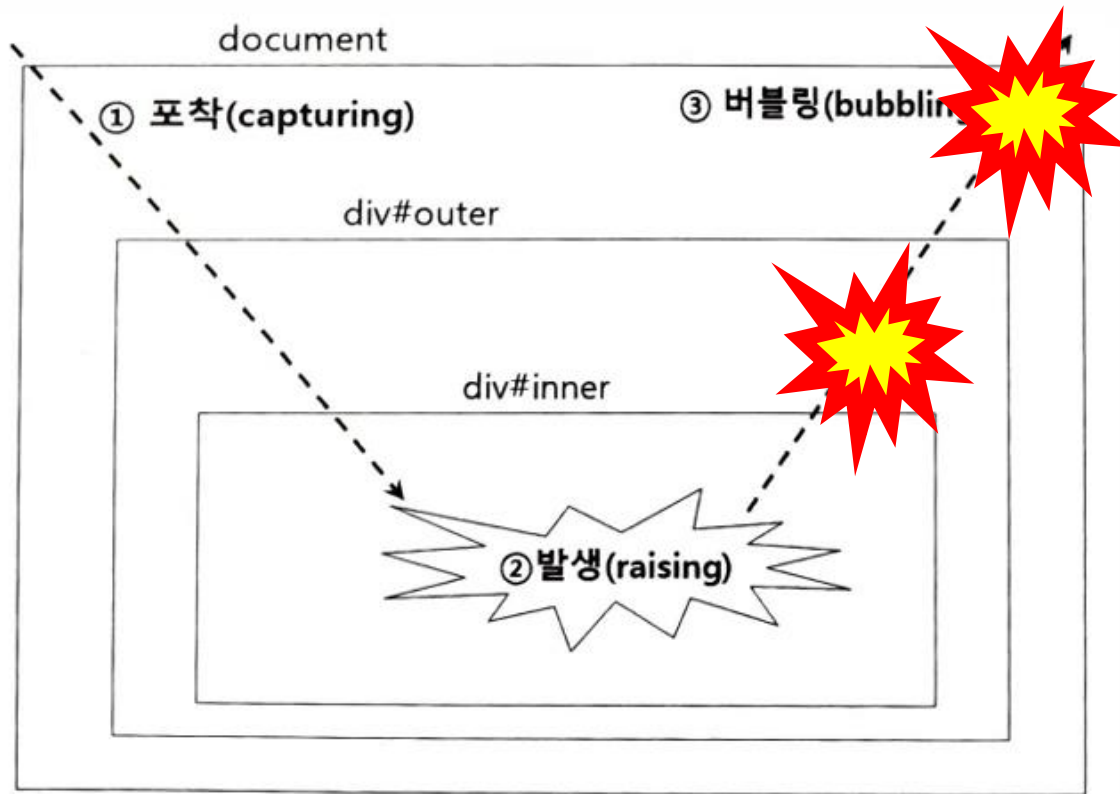
```
<body>
  <div id="app">
    <h1>{{ msg }}</h1>
    <input type="text" v-model="inputNum" />
    <button>확인</button>
  </div>
</body>
<script>
  const vueApp = Vue.createApp({
    name: 'App',
    data() {
      return {
        random: parseInt(Math.random() * 20),
        inputNum: 0,
        msg: '랜덤으로 생성된 숫자를 맞춰보세요',
      };
    },
  }).mount('#app');
</script>
```



이벤트

버블링

이벤트 버블링이 뭔가요!?



- 이걸 백문이불여일견!! 일단 코드로 보시죠!!



```
<body>
  <div id="app" style="background-color: skyblue">
    <h1>count : {{ count }}</h1>
    <div v-on:click="iDontWantToDiscount" style="background-color: orange">
      <h1>안쪽</h1>
      <button v-on:click="discount">클릭!</button>
    </div>
    <h1>바깥쪽</h1>
    <button v-on:click="discount">클릭!</button>
  </div>
</body>
<script>
  const vueApp = Vue.createApp({
    name: 'App',
    data() { return { count: 10 } },
    methods: {
      discount() {
        this.count--;
      },
      iDontWantToDiscount() {
        this.count--;
      },
    },
  }).mount('#app'); </script>
```

이벤트 버블링에 의해
button 태그의 부모 요소인
div 요소의 iDontWantToDiscount 도
같이 실행이 됩니다!

안쪽의 버튼을 클릭해서
discount 이벤트가 발생

count : 8

안쪽

클릭!

이벤트 버블링으로 인해
부모 div 요소에도 클릭 이벤트가 전달
→ iDontWantToDiscount 메서드도
실행 되어 count 가 추가로 1 감소

버튼 클릭 시,
discount 메서드 실행으로
count 가 1 감소

결과적으로 count 는 2가 감소하여
10 → 8 이 되어
원하던 결과와 다른 결과를 얻게 됨





```
<body>
  <div id="app" style="background-color: skyblue">
    <h1>count : {{ count }}</h1>
    <div v-on:click="iDontWantToDiscount" style="background-color: orange">
      <h1>안쪽</h1>
      <button v-on:click.stop="discount">클릭!</button>
    </div>
    <h1>바깥쪽</h1>
    <button v-on:click="discount">클릭!</button>
  </div>
</body>
<script>
  const vueApp = Vue.createApp({
    name: 'App',
    data() { return { count: 10 } },
    methods: {
      discount() {
        this.count--;
      },
      iDontWantToDiscount() {
        this.count--;
      },
    },
  }).mount('#app'); </script>
```

stop 키워드를 추가하여
e.stopPropagation 실행

count : 9



안쪽



클릭!



버튼 클릭 시,
discount 메서드 실행으로
count 가 1 감소



주요 이벤트 수식어 정리



- .stop
 - 이벤트 전파 중단
- .prevent
 - 해당 요소의 기본 이벤트 방지 / `event.preventDefault()` 동일
- .once
 - 이벤트 핸들러를 최대 한 번만 실행
- .passive
 - 이벤트의 기본 동작을 방지하지 않음 선언